

Practical-1

Aim: Study of phases in software development life cycle and software process models of project.

Software Development Life Cycle, SDLC for short, is a well-defined, structured sequence of stages in software engineering to develop the intended software product.

SDLC Activities

SDLC provides a series of steps to be followed to design and develop a software product efficiently. SDLC framework includes the following steps:



Communication : This is the first step where the user initiates the request for a desired software product. He contacts the service provider and tries to negotiate the terms. He submits his request to the service providing organization in writing.

Requirement Gathering : This step onwards the software development team works to carry on the project. The team holds discussions with various stakeholders from problem domain and tries to bring out as much information as possible on their requirements. The requirements are contemplated and segregated into user requirements, system requirements and functional requirements. The requirements are collected using a number of practices as given –

- studying the existing or obsolete system and software
- conducting interviews of users and developers
- referring to the database or
- collecting answers from the questionnaires.

Feasibility Study : After requirement gathering, the team comes up with a rough plan of software process. At this step the team analyzes if a software can be made to fulfill all requirements of the user and if there is any possibility of software being no more useful. It is found out, if the project is financially, practically and technologically feasible for the organization to take up. There are many algorithms available, which help the developers to conclude the feasibility of a software project.

System Analysis : At this step the developers decide a roadmap of their plan and try to bring up the best software model suitable for the project. System analysis includes Understanding of software product limitations, learning system related problems or changes to be done in existing systems beforehand, identifying and addressing the impact of project on organization and personnel etc. The project team analyzes the scope of the project and plans the schedule and resources accordingly.

Software Design : Next step is to bring down whole knowledge of requirements and analysis on the desk and design the software product. The inputs from users and information gathered in requirement gathering phase are the inputs of this step. The output of this step comes in the form of two designs; logical design and physical design. Engineers produce meta-data and data dictionaries, logical diagrams, data-flow diagrams and in some cases pseudo codes.

Coding : This step is also known as programming phase. The implementation of software design starts in terms of writing program code in the suitable programming language and developing error-free executable programs efficiently.

Testing : An estimate says that 50% of whole software development process should be tested. Errors may ruin the software from critical level to its own removal. Software testing is done while coding by the developers and thorough testing is conducted by testing experts at various levels of code such as module testing, program testing, product testing, in-house testing and testing the product at user's end. Early discovery of errors and their remedy is the key to reliable software.

Integration : Software may need to be integrated with the libraries, databases and other programs. This stage of SDLC is involved in the integration of software with outer world entities.

Implementation : This means installing the software on user machines. At times, software needs post-installation configurations at user end. Software is tested for portability and adaptability and integration related issues are solved during implementation.

Operation and Maintenance : This phase confirms the software operation in terms of more efficiency and less errors. If required, the users are trained on, or aided with the documentation on how to operate the software and how to keep the software operational. The software is maintained timely by updating the code according to the changes taking place in user end environment or technology. This phase may face challenges from hidden bugs and real-world unidentified problems.

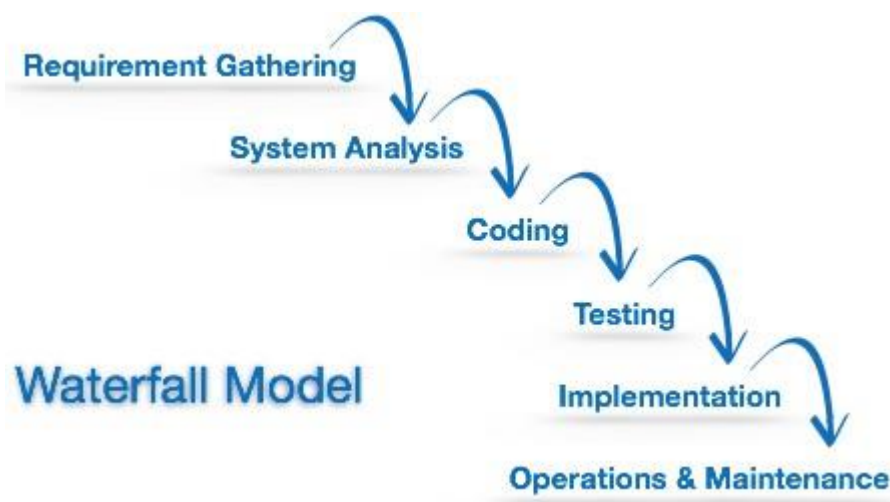
Disposition : As time elapses, the software may decline on the performance front. It may go completely obsolete or may need intense upgradation. Hence a pressing need to eliminate a major portion of the system arises. This phase includes archiving data and required software components, closing down the system, planning disposition activity and terminating system at appropriate end-of- system time.

Software Development Paradigm

The software development paradigm helps developer to select a strategy to develop the software. A software development paradigm has its own set of tools, methods and procedures, which are expressed clearly and defines software development life cycle. A few of software development paradigms or process models are defined as follows:

Waterfall Model

Waterfall model is the simplest model of software development paradigm. It says the all the phases of SDLC will function one after another in linear manner. That is, when the first phase is finished then only the second phase will start and so on.



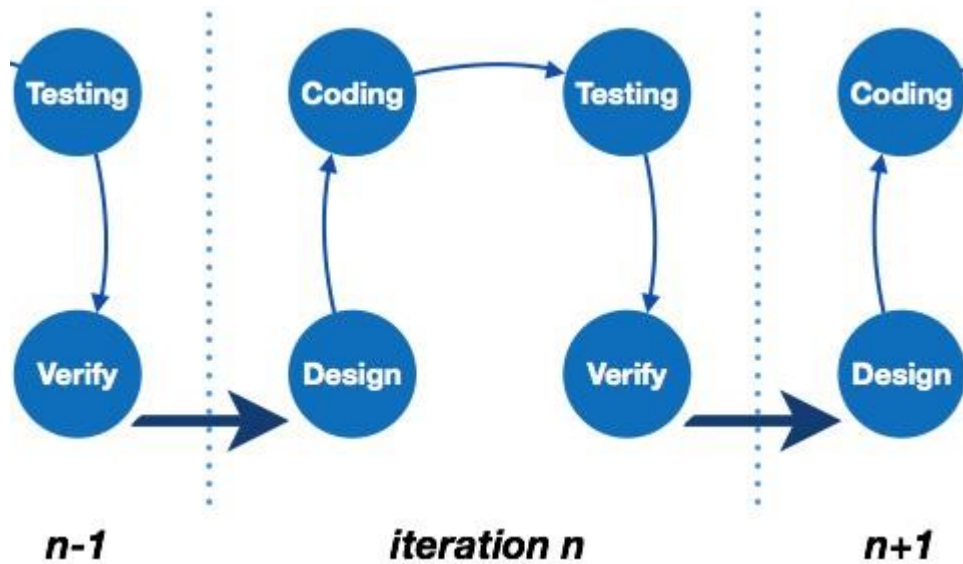
This model assumes that everything is carried out and taken place perfectly as planned in the previous stage and there is no need to think about the past issues that may arise in the next phase. This model does not work smoothly if there are some issues left at the previous step. The sequential nature of model does not allow us go back and undo or redo our actions.

This model is best suited when developers already have designed and developed similar software in the past and are aware of all its domain

Iterative Model

This model leads the software development process in iterations. It projects the process of development in cyclic manner repeating every step after every cycle of SDLC process.

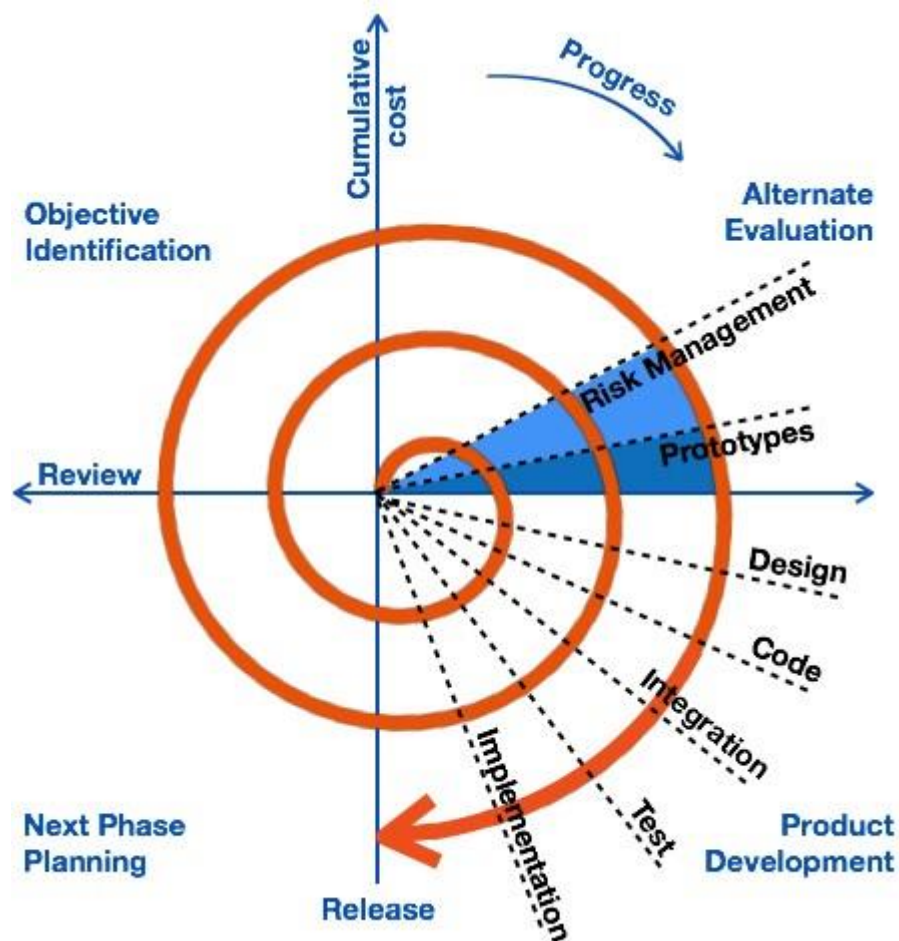
The software is first developed on very small scale and all the steps are followed which are taken into consideration. Then, on every next iteration, more features and modules are designed, coded, tested and added to the software. Every cycle produces a software, which is complete in itself and has more features and capabilities than that of the previous one.



After each iteration, the management team can do work on risk management and prepare for the next iteration. Because a cycle includes small portion of whole software process, it is easier to manage the development process but it consumes more resources.

Spiral Model

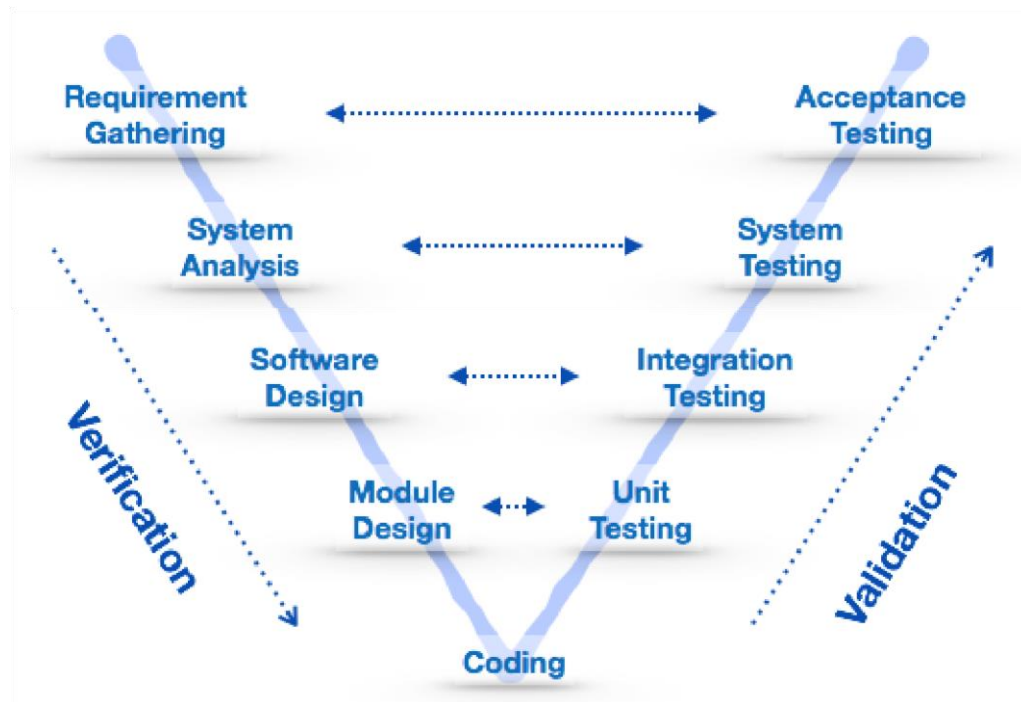
Spiral model is a combination of both, iterative model and one of the SDLC model. It can be seen as if you choose one SDLC model and combine it with cyclic process *iterativemodel*.



This model considers risk, which often goes un-noticed by most other models. The model starts with determining objectives and constraints of the software at the start of one iteration. Next phase is of prototyping the software. This includes risk analysis. Then one standard SDLC model is used to build the software. In the fourth phase of the plan of next iteration is prepared.

V – model

The major drawback of waterfall model is we move to the next stage only when the previous one is finished and there was no chance to go back if something is found wrong in later stages. V-Model provides means of testing of software at each stage in reverse manner.



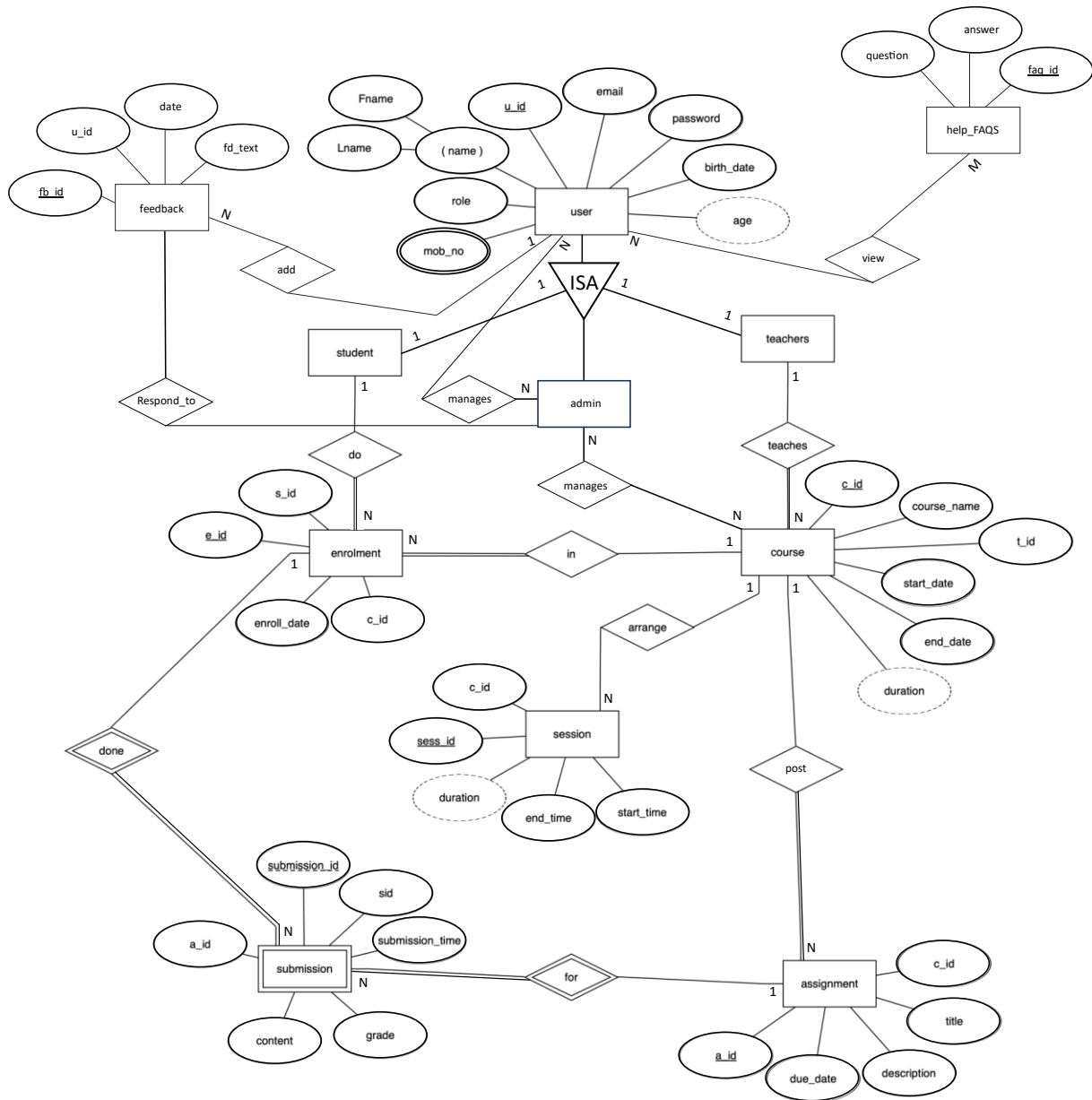
At every stage, test plans and test cases are created to verify and validate the product according to the requirement of that stage. For example, in requirement gathering stage the test team prepares all the test cases in correspondence to the requirements. Later, when the product is developed and is ready for testing, test cases of this stage verify the software against its validity towards requirements at this stage.

This makes both verification and validation go in parallel. This model is also known as verification and validation model.

Practical-3

Aim: To perform Data Modeling: E-R Diagram

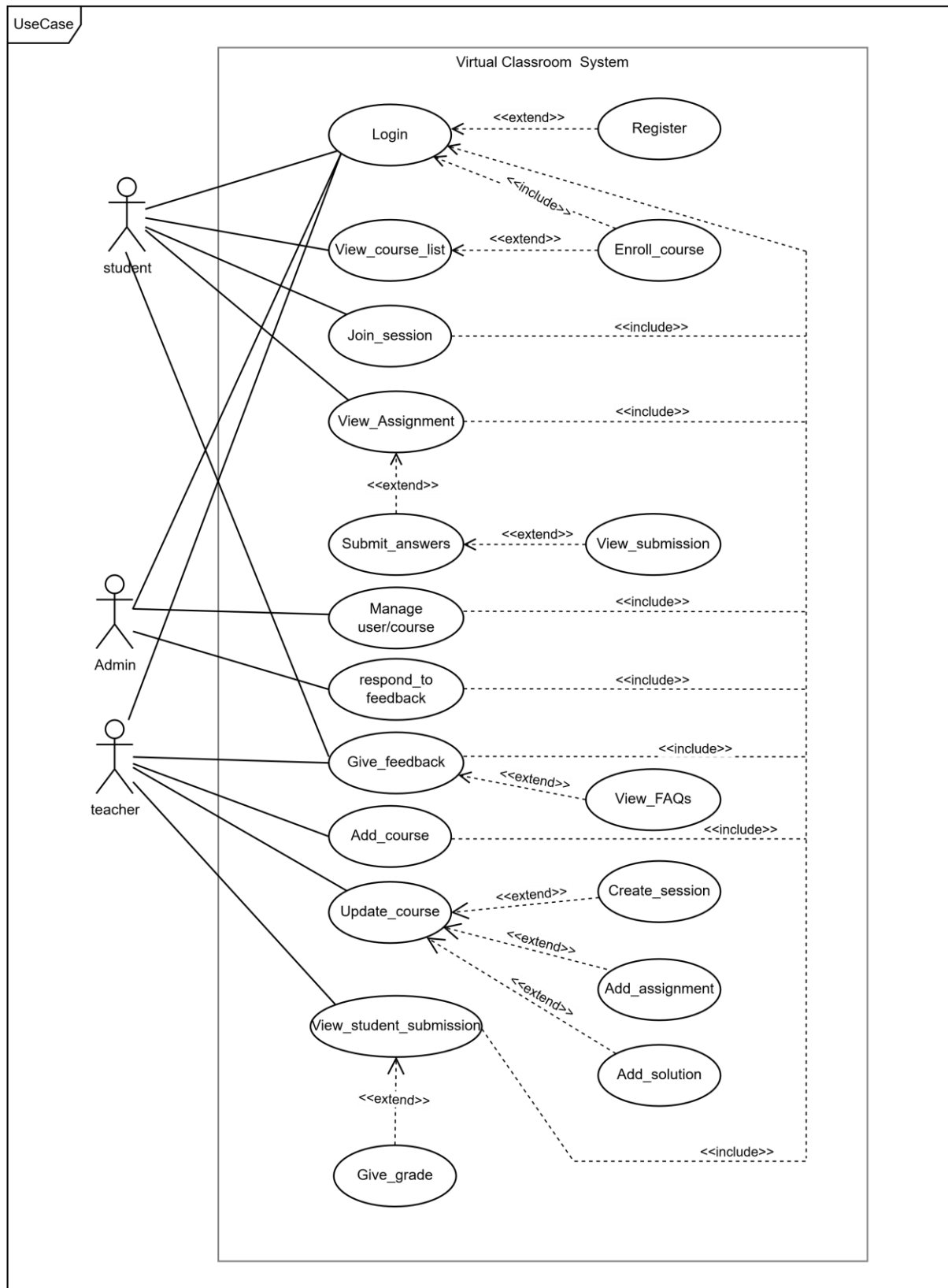
Project Name: Virtual Classroom System



Practical-4

Aim: To perform the Scenario Based Modeling : Use Case Diagram

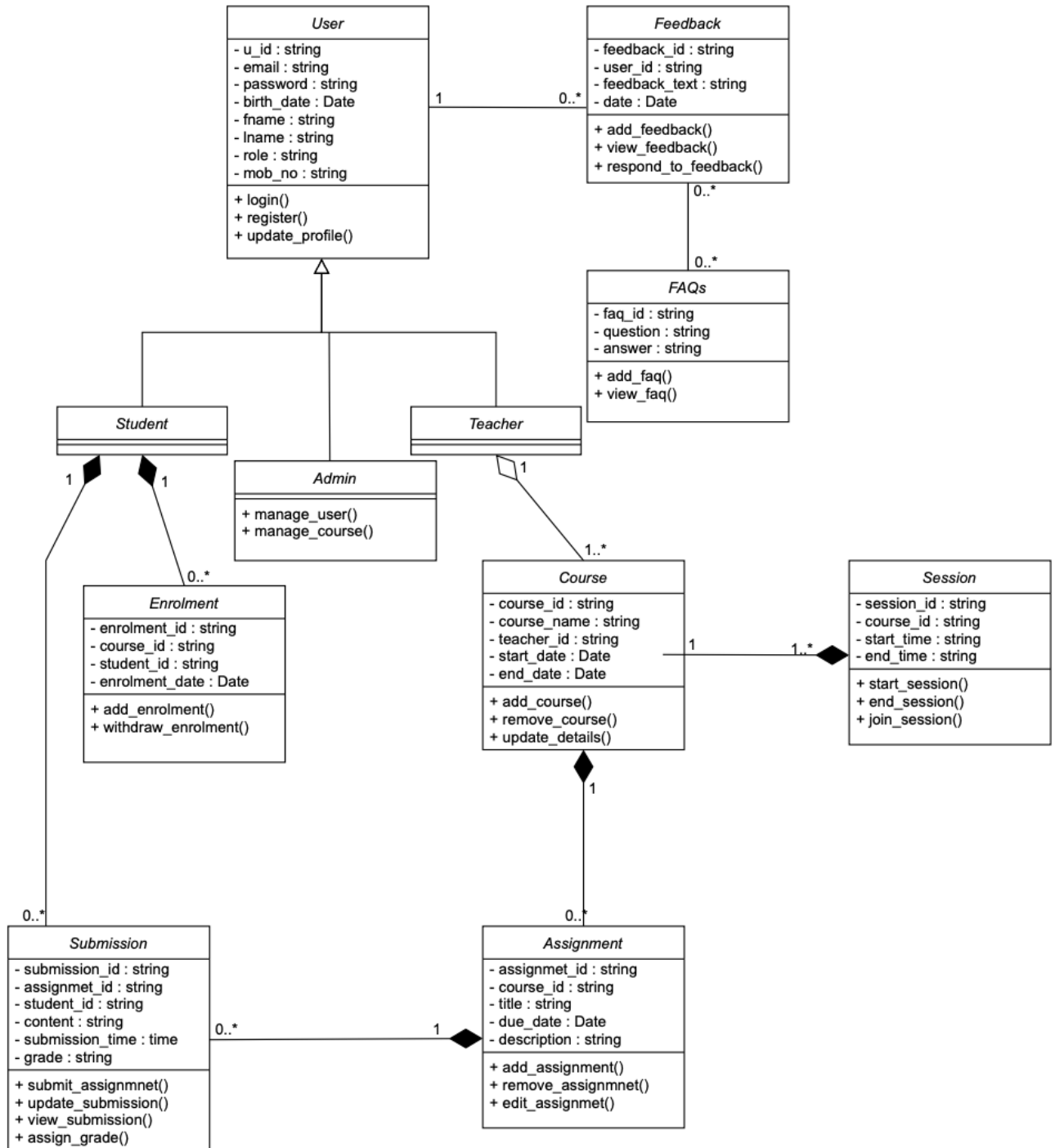
Project Name: Virtual Classroom System



Practical-5

Aim: To perform Class Based Modeling : Class Diagram

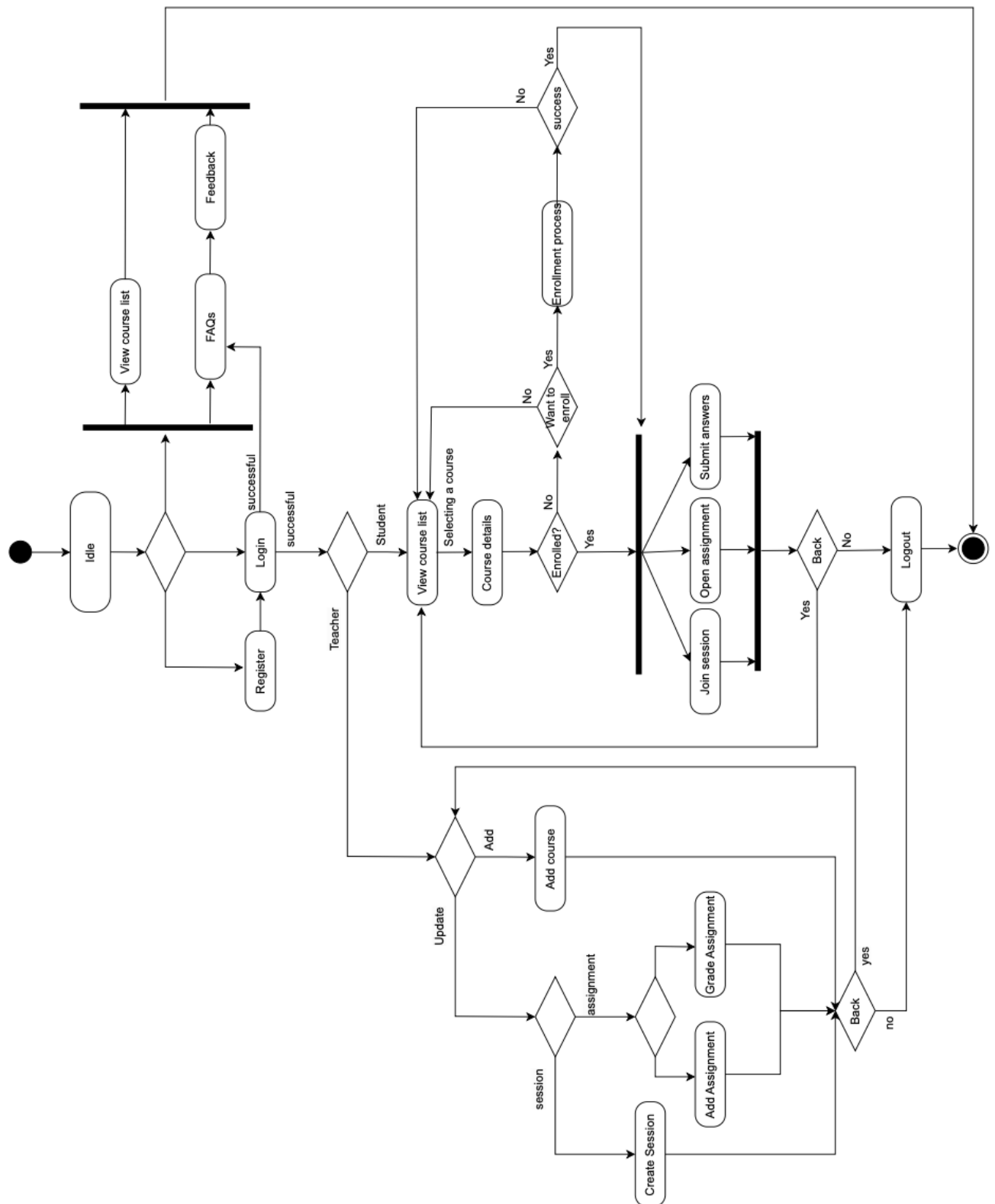
Project Name: Virtual Classroom System



Practical-6

Aim: To draw the Behavioral View Diagram: State Diagram

Project Name: Virtual Classroom System

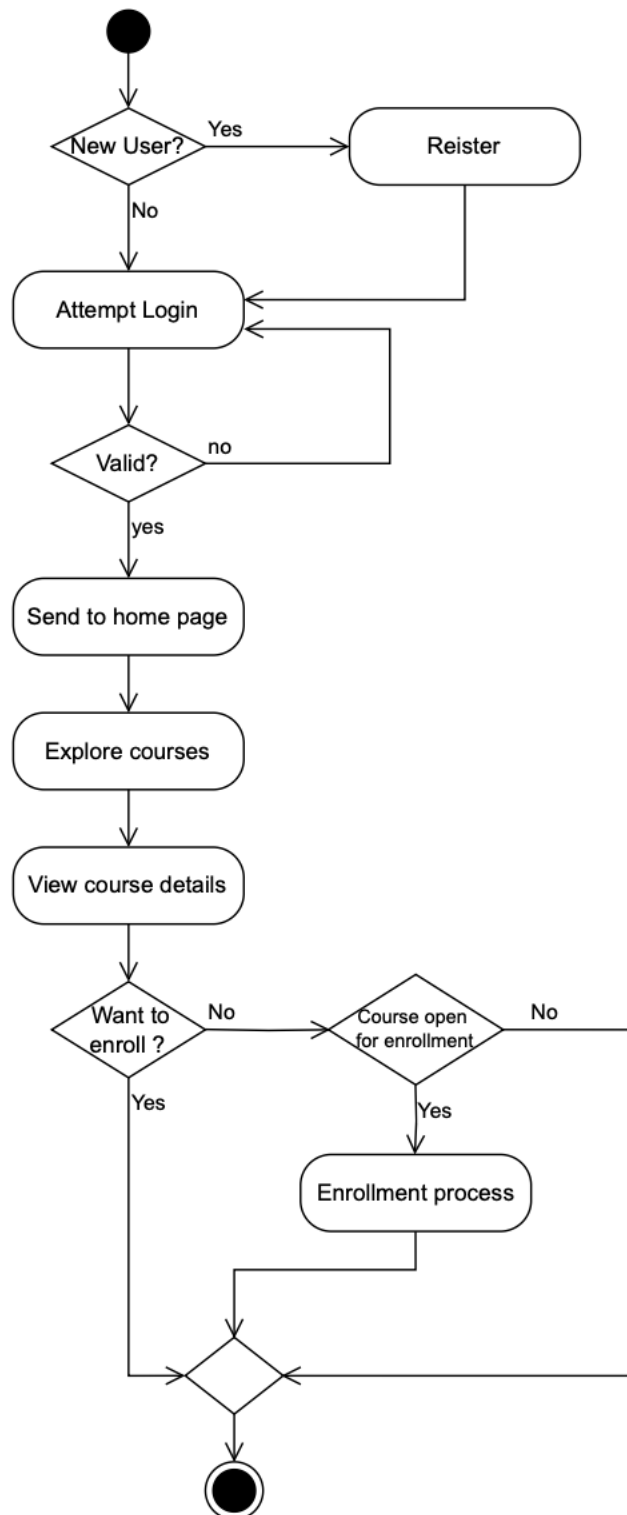


Practical-7

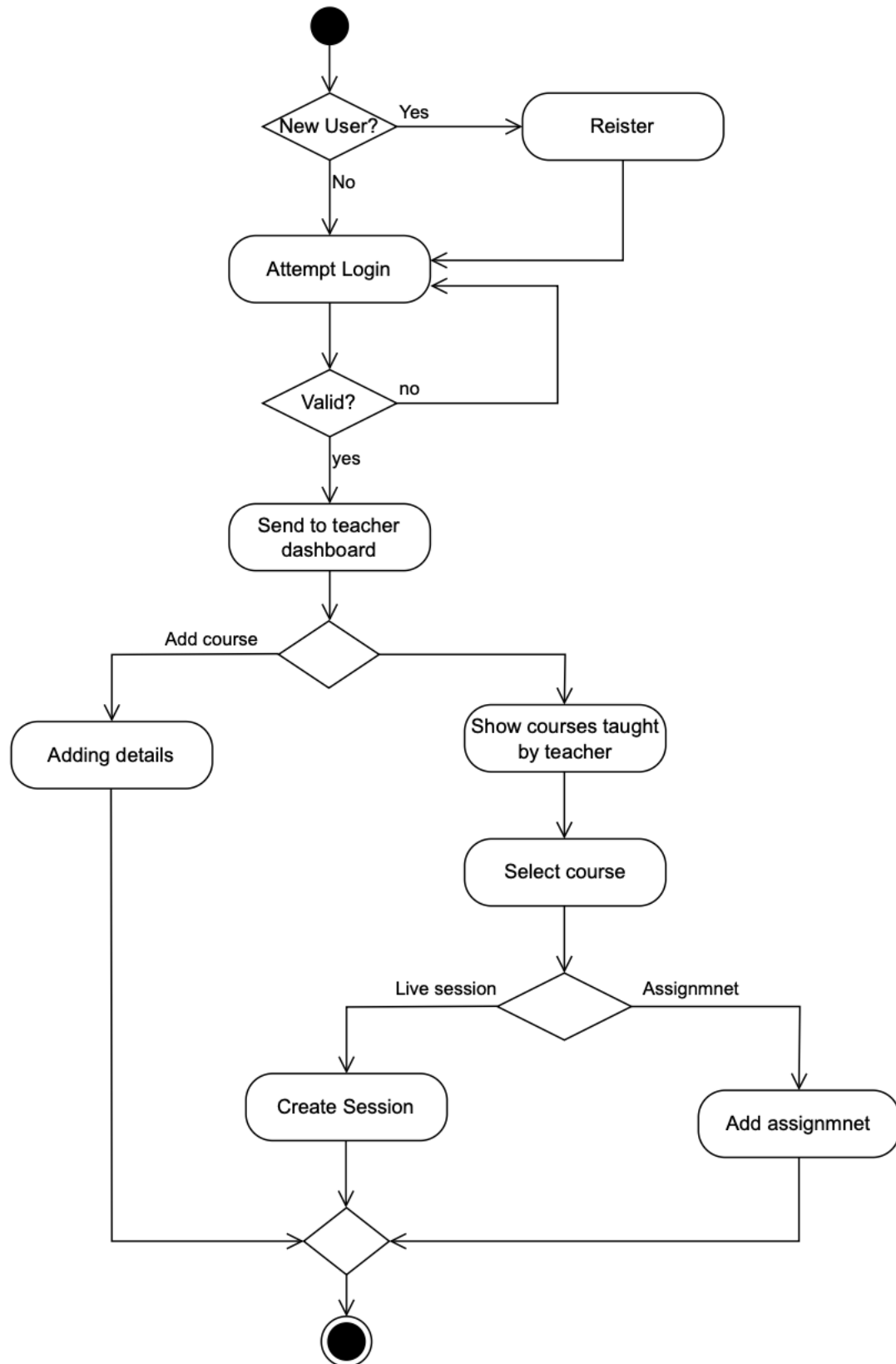
Aim: To draw the Behavioral View Diagram: Activity Diagram

Project Name: Virtual Classroom System

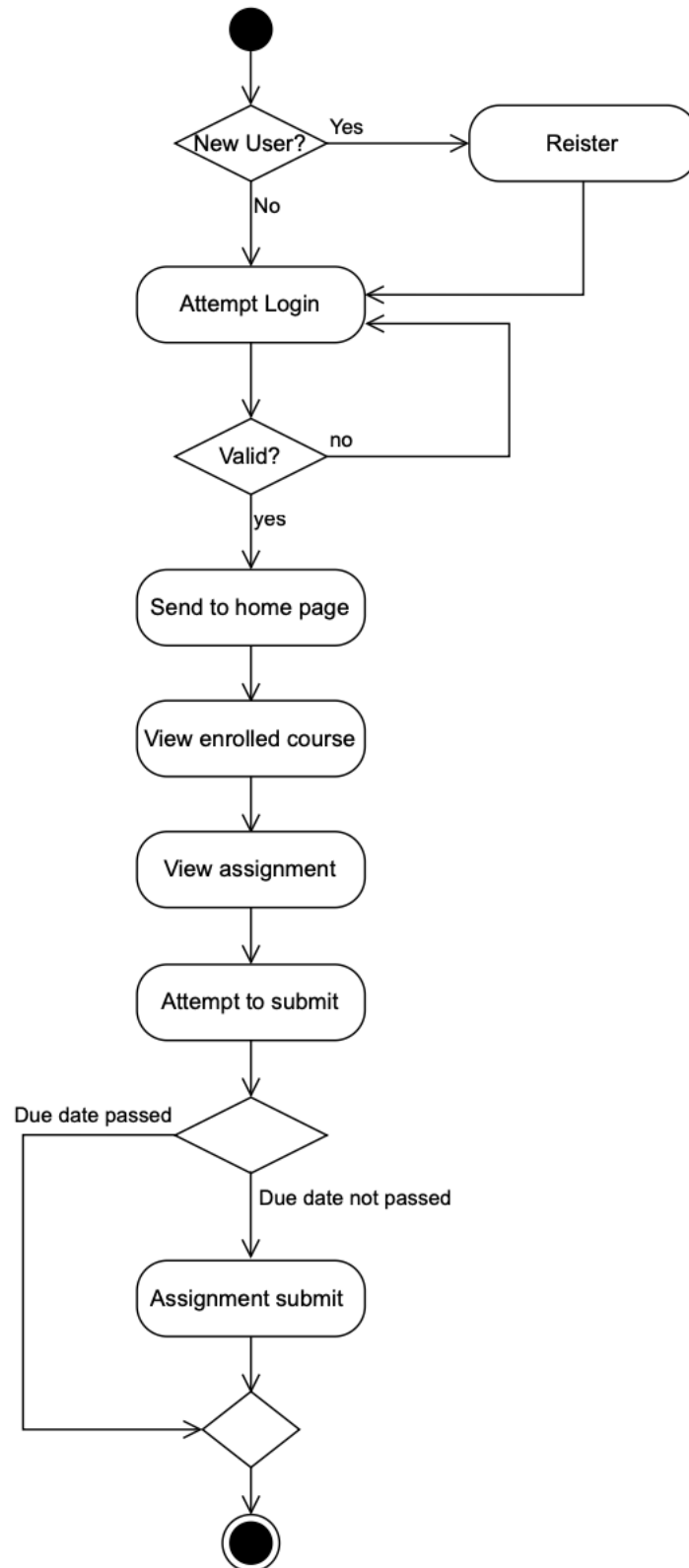
1. Activity: Enrolling into Course



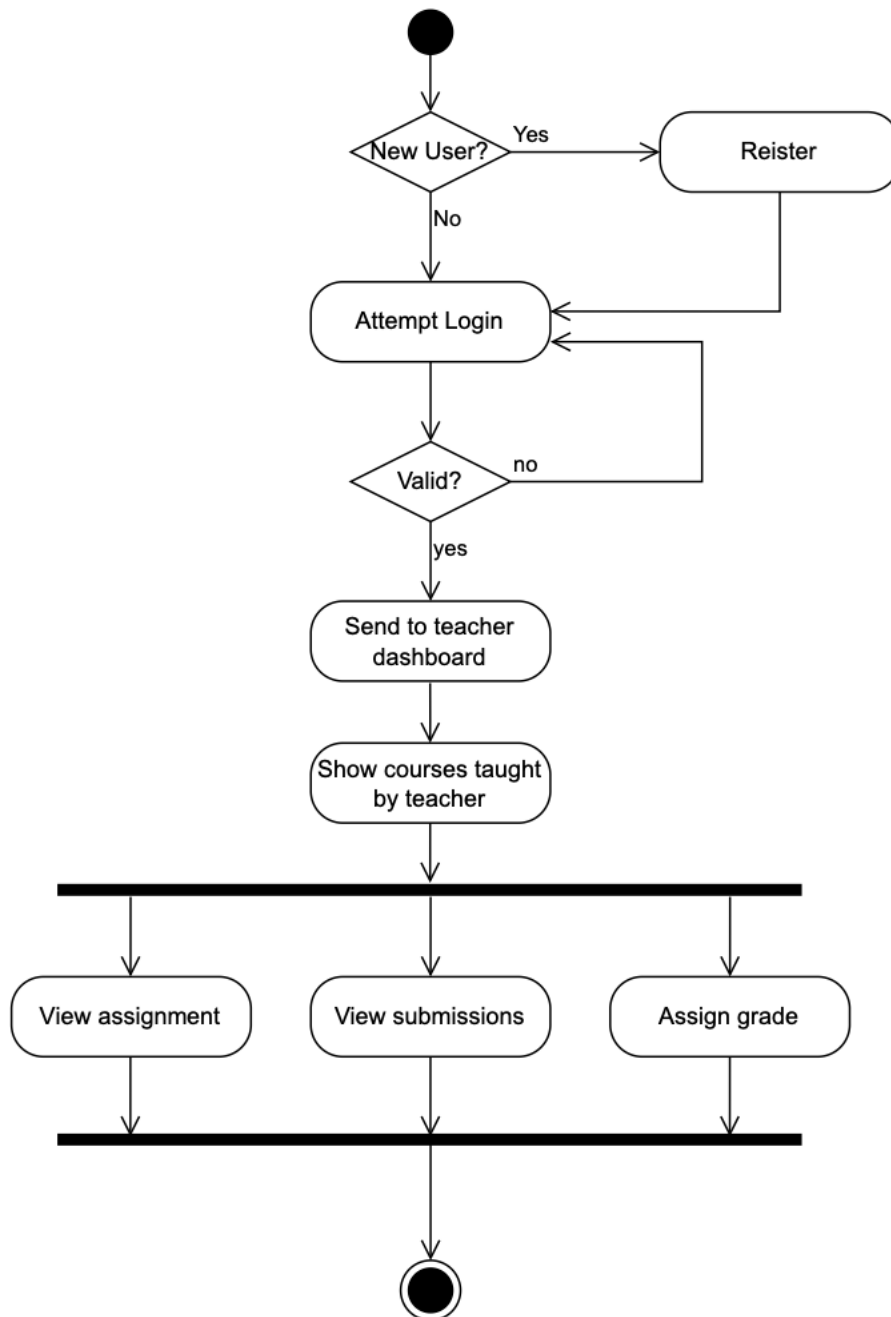
2. Activity: Enrolling into Course



3. Activity: Adding assignment submission



4. Activity: Assigning grades to assignment

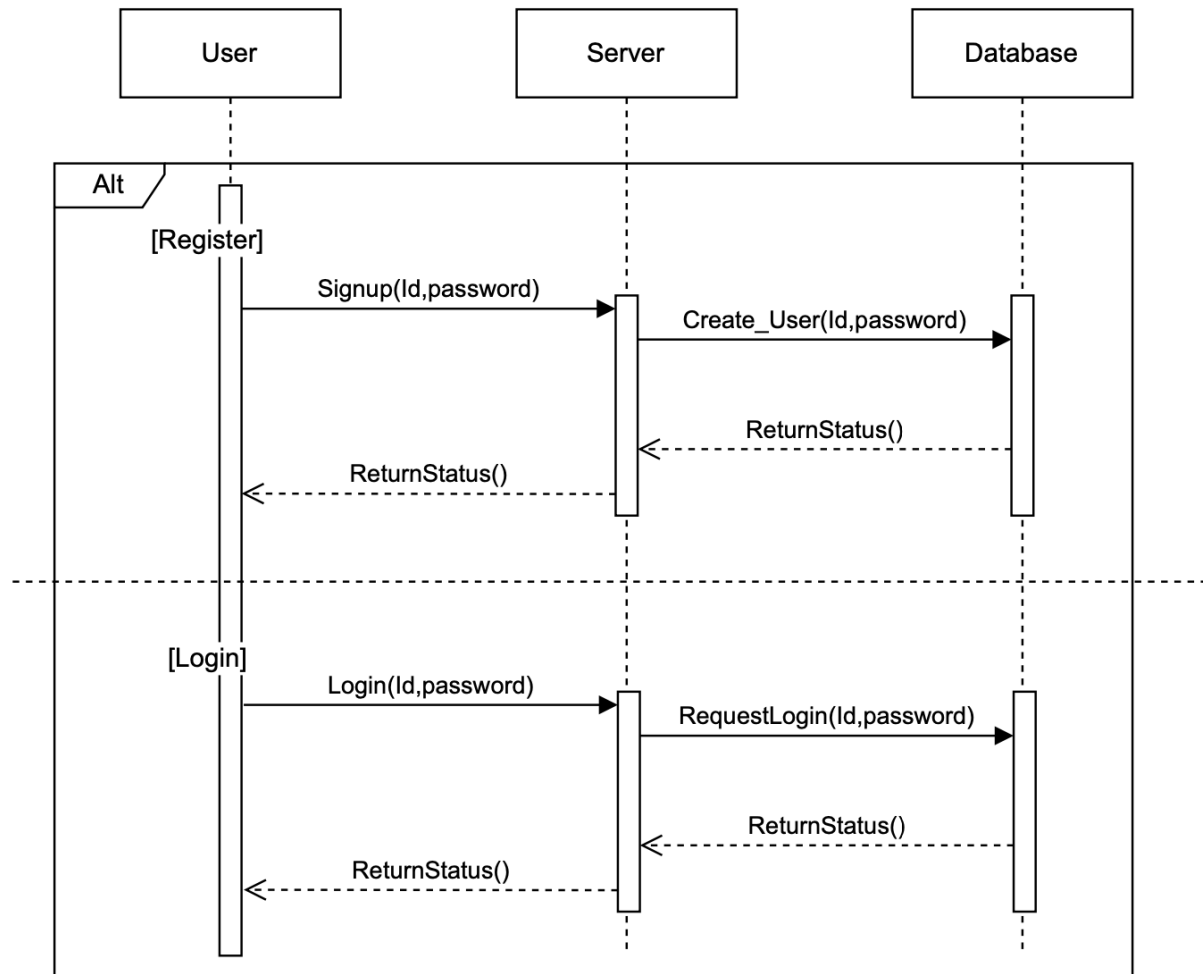


Practical-8

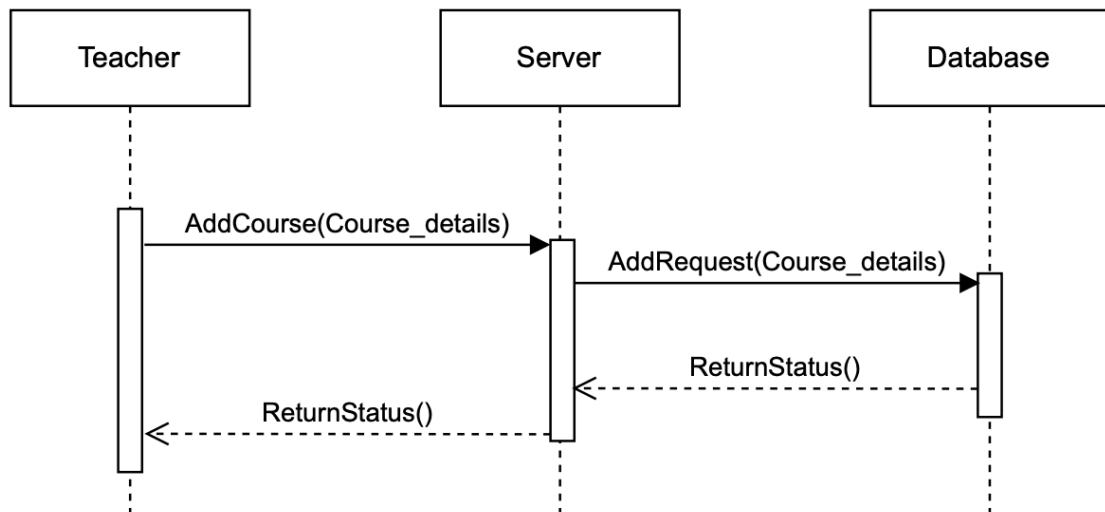
Aim: To draw the Behavioral View Diagram: Sequence Diagram

Project Name: Virtual Classroom System

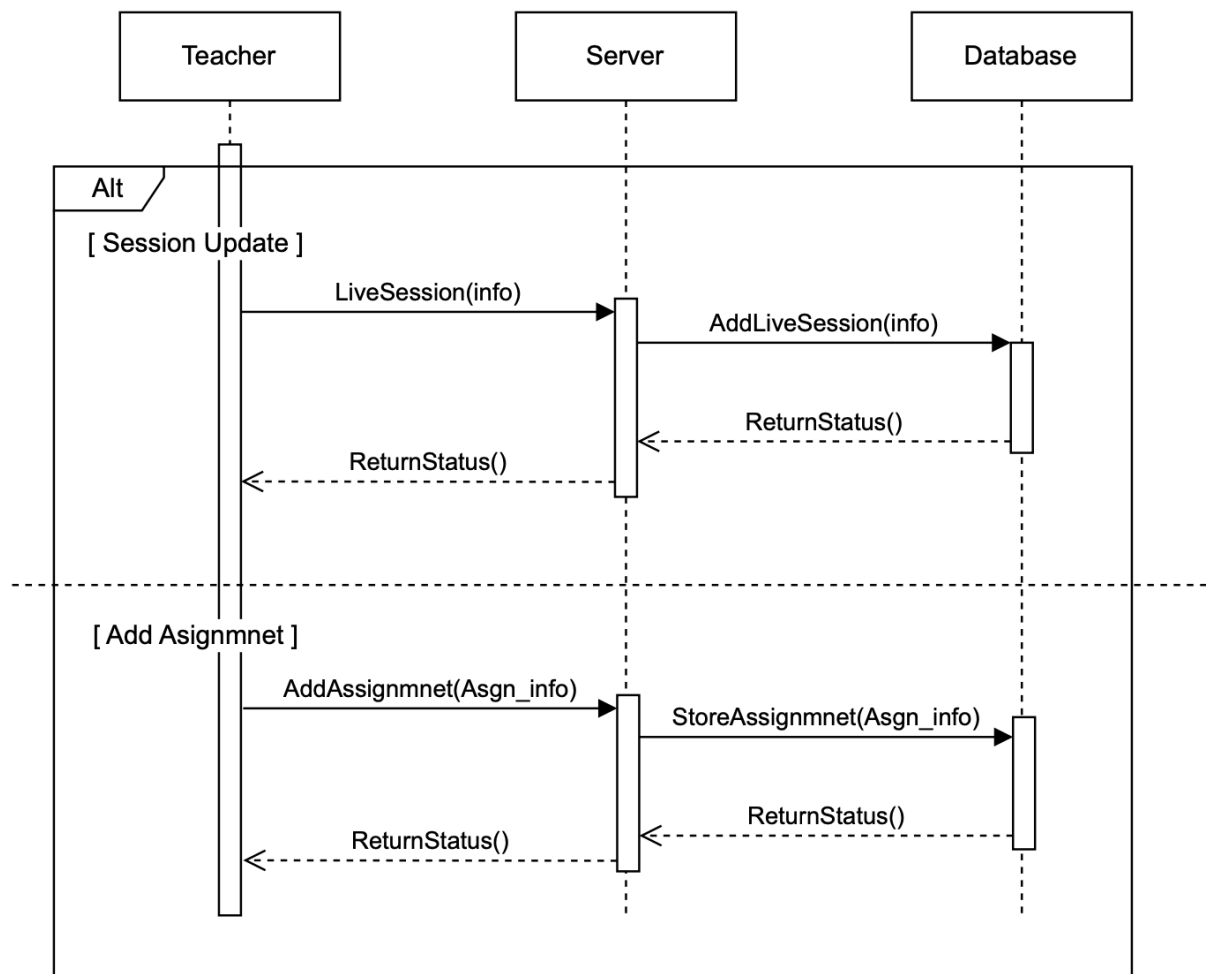
1. Sequence diagram for Login/Register



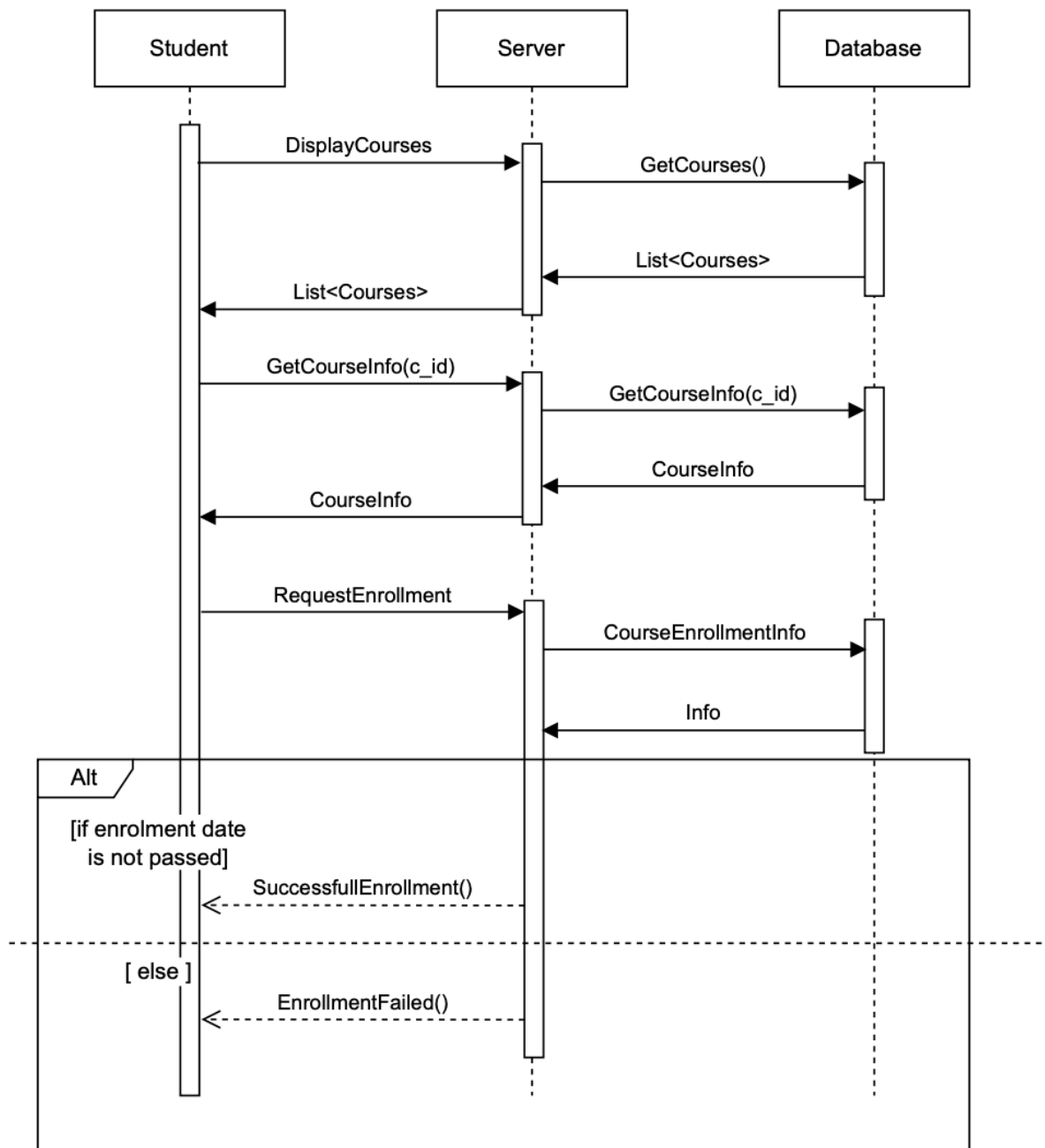
2. Sequence diagram for Adding Course



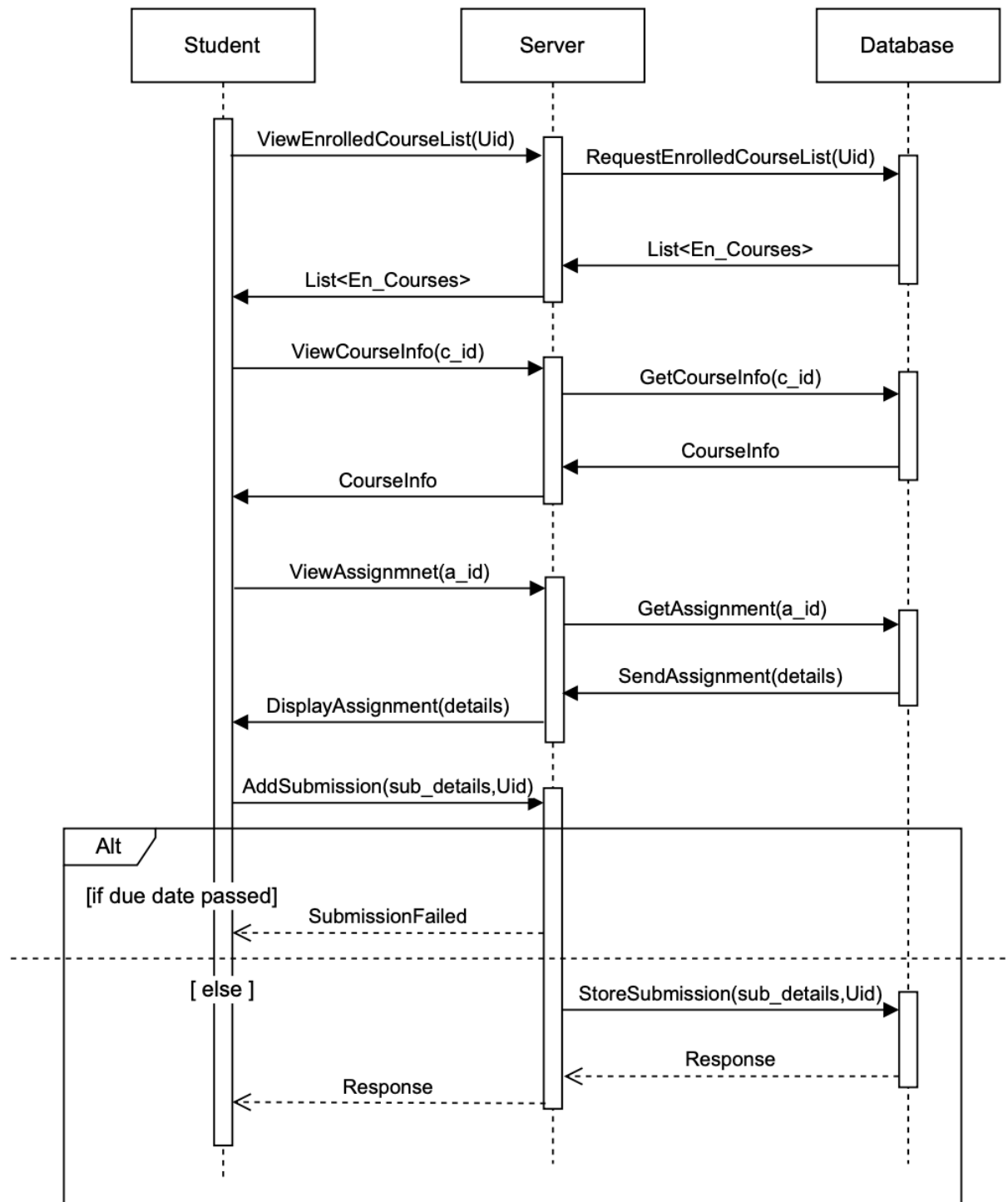
3. Sequence diagram for Updating Course



4. Sequence diagram for Enrolling into Course



5. Sequence diagram for Adding Submission

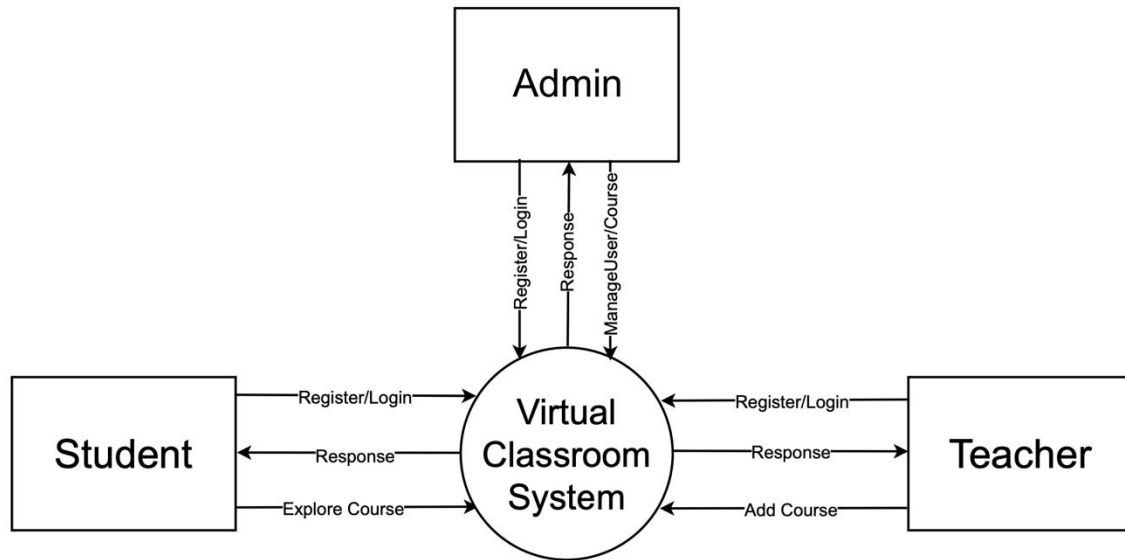


Practical-9

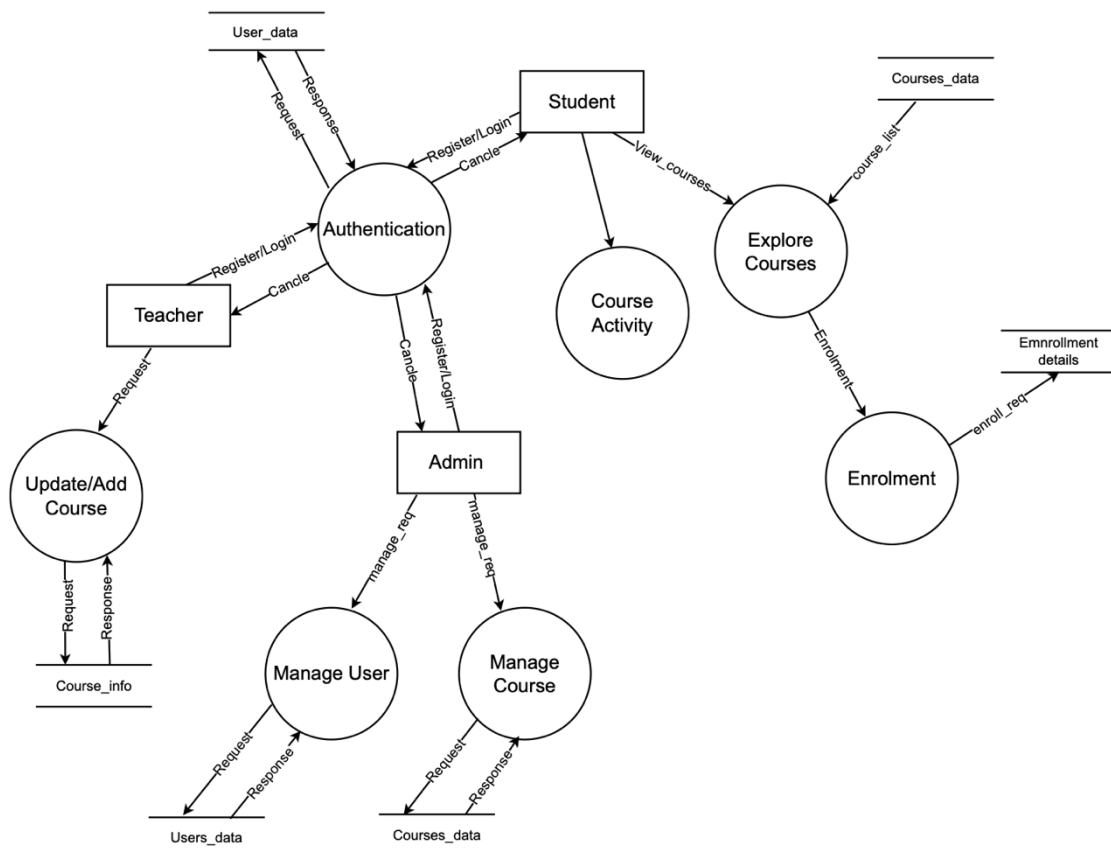
Aim: Flow-Oriented Modeling: Data Flow Diagram

Project Name: Virtual Classroom System

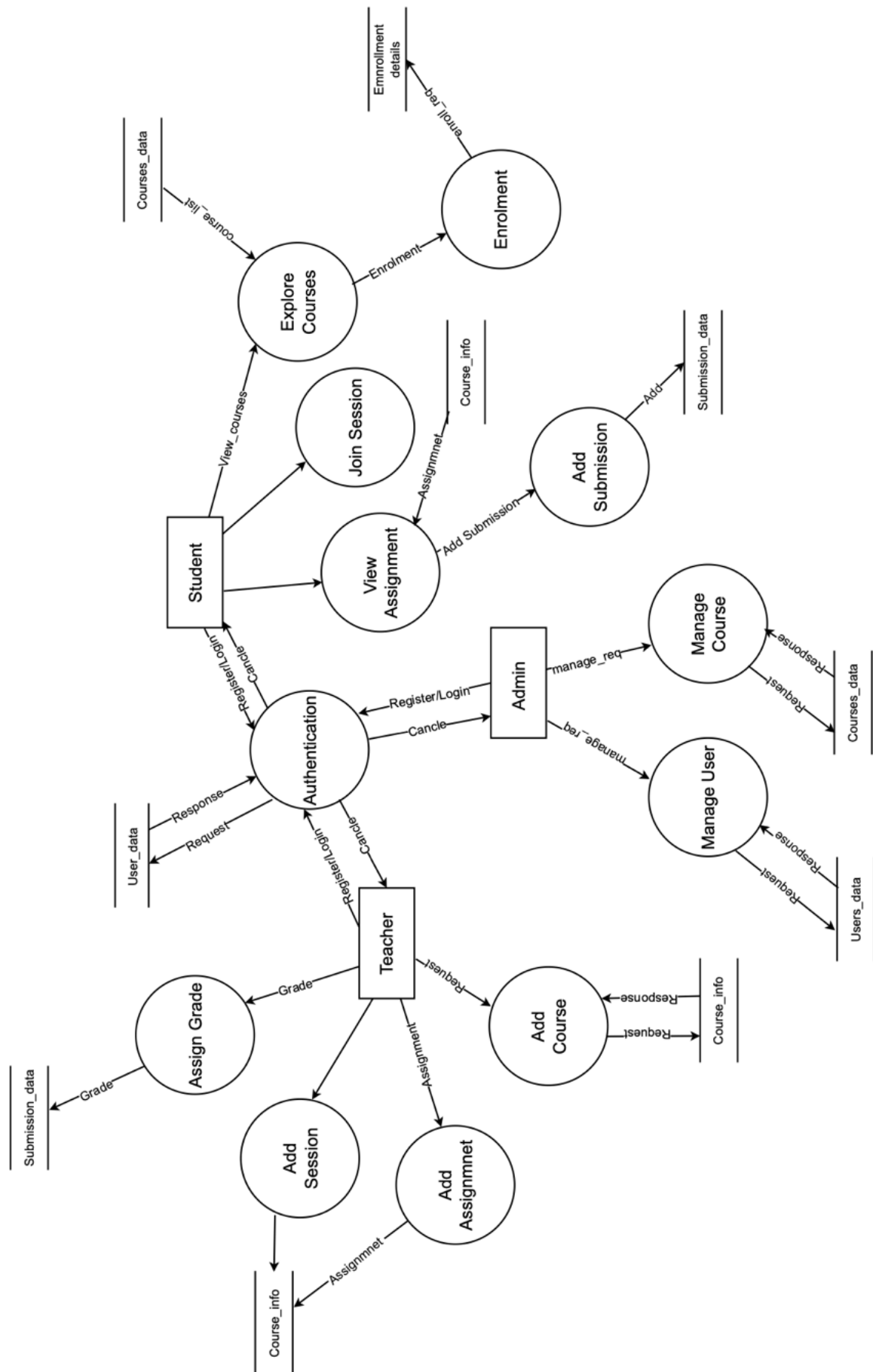
DFD Level 0



DFD Level 1



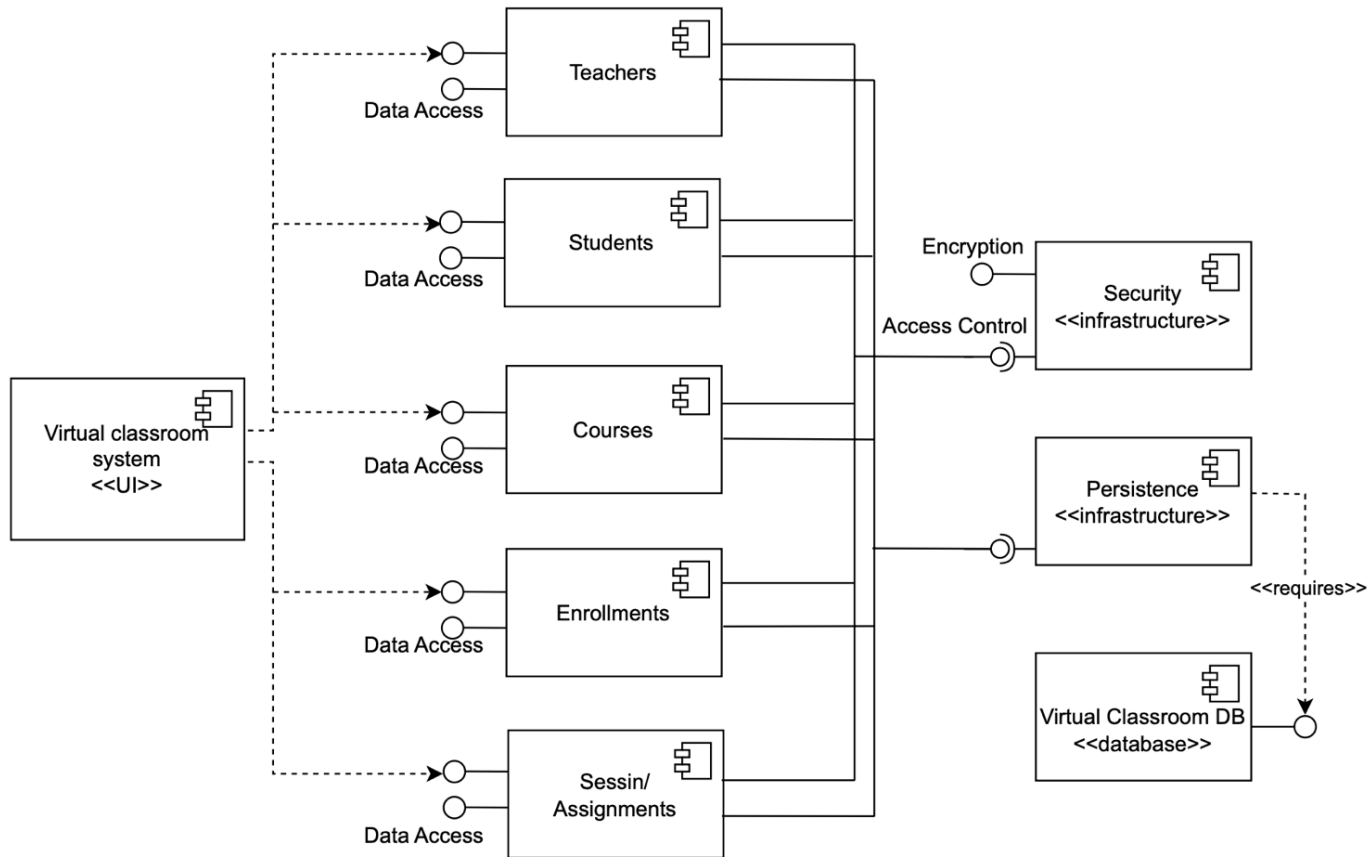
DFD Level 2



Practical-10

Aim: To draw the Component diagram.

Project Name: Virtual Classroom System



Practical-11 : To Determine the Schedule of Software: TimeLine Chart

Project Name: Virtual Classroom System

[illegible]

Practical-12 :

Aim: Study to Find out the Cost of CAD Software: Using LOC & FP Approach

Estimation table for the LOC methods for CAD software

Function	Estimated LOC
User interface and control facilities (UICF)	2,300
Two-dimensional geometric analysis (2DGA)	5,300
Three-dimensional geometric analysis (3DGA)	6,800
Database management (DBM)	3,350
Computer graphics display facilities (CGDF)	4,950
Peripheral control function (PCF)	2,1000
Design analysis modules (DAM)	8,400
<i>Estimated lines of code</i>	33,200

organizational average productivity for systems of this type is 620 LOC/pm.

burdened labor rate of \$8000 per month.

So, cost per line of code approximately = $8000/620$

$$\approx \$13$$

total estimated project cost = Estimated line of code * cost per line of code

$$= 33200 * 13$$

$$= \$431,600$$

$$\approx \$431,000$$

estimated effort = Estimated line of code / organizational average productivity(LOC/pm)

$$= 33200 / 620$$

$$\approx 54 \text{ person-month}$$

Estimation table for the FP methods for CAD software

Information domain value	Opt.	Likely	Pess.	Est. count	Weight	FP count
Number of external inputs	20	24	30	24	4	97
Number of external outputs	12	15	22	16	5	78
Number of external inquiries	16	22	28	22	5	88
Number of internal logical files	4	4	5	4	10	42
Number of external logical files	2	2	3	2	7	15
Count total						320

Factor	Value
Backup and recovery	4
Data communications	2
Distributed processing	0
Performance critical	4
Existing operating environment	3
Online data entry	4
Input transaction over multiple screens	5
Master files updated online	3
Information domain values complex	5
Internal processing complex	5
Code designed for reuse	4
Conversion/installation in design	3
Multiple installations	5
Application designed for change	5
$\Sigma(F)$	52

organizational average productivity for systems of this type is 6.5 FP/pm.

burdened labor rate of \$8000 per month.

$$FP_{\text{estimated}} = \text{count total} \times [0.65 + 0.01 * \Sigma(F)] = 375$$

So, cost per line of code approximately = $8000/6.5$

$$\approx \$1230$$

total estimated project cost = Estimated FP * cost per FP

$$= 320 * 1230$$

$$= \$461,250$$

$$\approx \$461,000$$

$$\begin{aligned}\text{estimated effort} &= \text{Estimated FP} / \text{organizational average productivity(FP/pm)} \\ &= 375 / 6.5 \\ &\approx 58 \text{ person-month}\end{aligned}$$

Practical-13

Aim: Designing Test Suites.

Project Name: Virtual Classroom System

TC-01

Test Case ID	TC-01
Module to be tested	Login Page
Assumptions	---
Test Data	Check When passing a correct username and invalid password
Test Steps	1. Enter valid username 2. Enter incorrect password 3. Click on Login Button
Expected results	User should not log in and should show proper error message
Result	Pass
Comments	On entering an invalid password, user not able to login application. It is required to enter the correct username and password.

TC-02

Test Case ID	TC-02
Module to be tested	Login Page
Assumptions	---
Test Data	Check when pass correct email (username) and password
Test Steps	1. Enter valid username 2. Enter valid password 3. Click on Login Button
Expected results	User should be successfully logged in and redirected to the home page/dashboard.
Result	Pass
Comments	On entering a valid username and password user will able to log in to the application.

TC-03

Test Case ID	TC-03
Module to be tested	Registration Page
Assumptions	User is a new user.
Test Data	Check registration with valid details.
Test Steps	1. Navigate to the registration page. 2. Enter valid registration details (e.g., username, email, password, etc.). 3. Click on the Register button.
Expected results	User should be successfully registered and redirected to the login page.
Result	Pass
Comments	Upon successful registration, the user should be able to log in with the registered credentials.

TC-04

Test Case ID	TC-04
Module to be tested	Registration Page
Assumptions	User is a new user.
Test Data	Check registration with invalid details.
Test Steps	1. Navigate to the registration page. 2. Enter invalid registration details (e.g., incomplete fields, invalid email format, etc.) 3. Click on the Register button.
Expected results	User should not be able to register and should see an error message indicating the reason for registration failure.
Result	Pass
Comments	The registration process should validate user input and reject invalid registrations, providing clear feedback to the user on the registration failure.

TC-05

Test Case ID	TC-05
Module to be tested	Enrollment Process
Assumptions	User is authenticated and logged in.
Test Data	Enrollment with valid details.
Test Steps	1. Navigate to the enrollment page. 2. Select the desired course from the list of available courses. 3. Click on the Enroll button.
Expected results	User should not be able to register and should see an error message indicating the reason for registration failure.
Result	Pass
Comments	Upon successful enrollment, the user should have access to course Assignments and other features related to the enrolled course.

TC-06

Test Case ID	TC-06
Module to be tested	Enrollment Process
Assumptions	User is authenticated and logged in.
Test Data	Enrollment after enrollment date has passed.
Test Steps	1. Navigate to the enrollment page. 2. Select the desired course from the list of available courses. 3. Click on the Enroll button.
Expected results	User should see a message indicating that enrollment for the selected course is closed due to the passing of the enrollment date, including any alternative options or next steps.
Result	Pass
Comments	Ensure that the system handles the scenario of enrollment closure gracefully, providing users with informative feedback and guidance.

TC-07

Test Case ID	TC-07
Module to be tested	Add Course
Assumptions	User is authenticated and logged in as Teacher.
Test Data	Check adding a new course.
Test Steps	1. Navigate to the course creation page. 2. Enter valid course details (e.g., title, description, etc.). 3. Click on the Add Course button.
Expected results	New course should be successfully added to the system.
Result	Pass
Comments	Upon successful addition, the new course should be available for enrollment and other operations.

TC-08

Test Case ID	TC-08
Module to be tested	Assignment Submission Page
Assumptions	User is enrolled in a course with assignments.
Test Data	Submission of assignment before the due date.
Test Steps	1. Navigate to the assignment submission page for the enrolled course. 2. Select the assignment to submit. 3. Upload a valid assignment file. 4. Click on the Submit button.
Expected results	Assignment should be successfully submitted.
Result	Pass
Comments	Upon successful submission, the assignment should be recorded for evaluation by the instructor.

TC-09

Test Case ID	TC-09
Module to be tested	Assignment Submission Page
Assumptions	User is enrolled in a course with assignments.
Test Data	Submission of assignment after the due date.
Test Steps	1. Navigate to the assignment submission page for the enrolled course. 2. Select the assignment to submit. 3. Upload a valid assignment file. 4. Click on the Submit button.
Expected results	Assignment should not be accepted. User should see a message indicating that submission is not allowed as the assignment due date has passed.
Result	Pass
Comments	The system should provide clear and informative feedback to the user regarding the submission status.

TC-10

Test Case ID	TC-10
Module to be tested	Session Joining Page
Assumptions	User is enrolled in a course with the session.
Test Data	Joining a session within the specified time frame.
Test Steps	1. Navigate to the session joining page for the respective course. 2. Select the desired session from the list of available sessions. 3. Click on the Join button.
Expected results	User should be successfully joined into the selected session and redirected to the session room.
Result	Pass
Comments	Ensure that the system properly handles session joining and redirects the user to the correct session room upon successful joining.

TC-11

Test Case ID	TC-11
Module to be tested	Session Joining Page
Assumptions	User is enrolled in a course with the session.
Test Data	Attempting to join a session after it has ended or canceled.
Test Steps	1. Navigate to the session joining page for the respective course. 2. Select a session that has either ended, been canceled. 3. Click on the Join button.
Expected results	User should see a message indicating that joining the session is not possible due to the session has ended or been canceled.
Result	Pass
Comments	Ensure that the system provides informative feedback to the user when attempting to join a session after it has ended or been canceled. The message should guide the user on how to proceed or provide alternatives if available.

Practical-14

Aim: Analyze the Risk related to Project

Project Name: Virtual Classroom System

1. Project Risks:

- Scope Creep: Expansion of project scope beyond initial requirements, leading to delays, increased costs, and potential conflicts with project timelines.
- Resource Constraints: Insufficient human resources, expertise, or budget allocation, hindering the timely completion of project milestones and deliverables.
- Communication Breakdown: Ineffective communication among project stakeholders, team members, or external partners, resulting in misunderstandings, delays, and alignment issues.

2. Technical Risks:

- System Compatibility: Compatibility issues with various devices, operating systems, web browsers, or internet connections used by students and instructors, affecting usability and accessibility.
- Performance Bottlenecks: Inadequate system scalability or optimization, leading to performance degradation, latency issues, or system crashes during peak usage periods.
- Data Security: Vulnerabilities in data storage, transmission, or authentication mechanisms, posing risks of data breaches, unauthorized access, or loss of sensitive information.

3. Business Risks:

- Market Competition: Intense competition from existing virtual classroom platforms or new market entrants, challenging market penetration, user acquisition, and revenue growth.
- Monetization Challenges: Difficulty in establishing a viable revenue model, monetizing user engagement, or achieving profitability amidst market uncertainties and changing user preferences.
- Financial Viability: Financial constraints, funding limitations, or budget overruns impacting project sustainability, resource allocation, and operational stability.

4. Known Risks:

- Technology Dependencies: Reliance on third-party APIs, libraries, or external services for critical functionalities, with the risk of service interruptions, version changes, or compatibility issues.
- Regulatory Compliance: Compliance requirements with data protection regulations (e.g., GDPR, CCPA) and educational standards (e.g., FERPA), necessitating adherence to legal obligations and industry guidelines.

- User Adoption: Challenges in driving user adoption and engagement, overcoming resistance to change, and addressing user preferences, expectations, and feedback effectively.

5. Predictable Risks:

- Development Delays: Anticipated delays in software development, testing, or deployment phases due to technical complexities, resource constraints, or unforeseen obstacles.
- Technical Debt: Accumulation of technical debt resulting from shortcuts, compromises, or deferred tasks during the development process, requiring future remediation and refactoring efforts.
- Market Demand Shifts: Changes in market demand, educational trends, or learning preferences influencing user requirements, feature prioritization, and product roadmap adjustments.